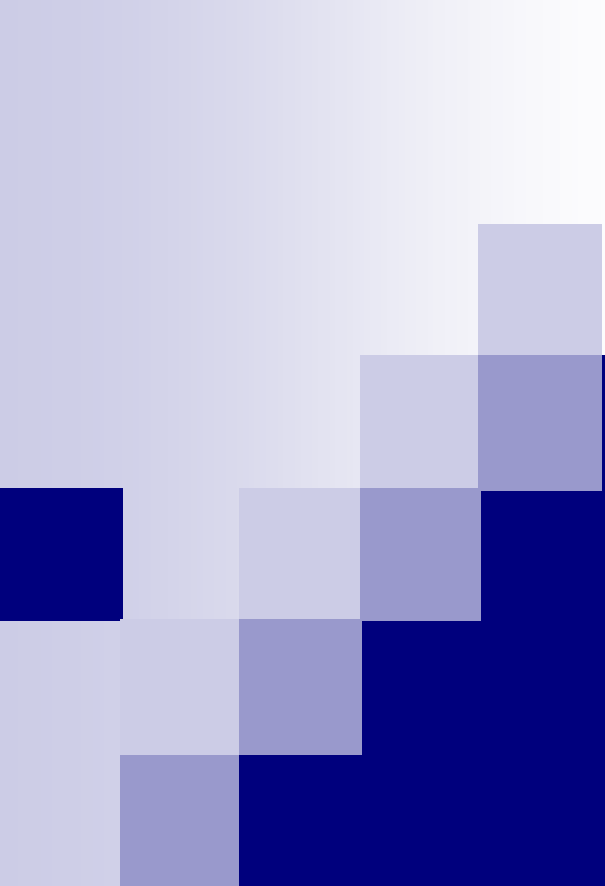


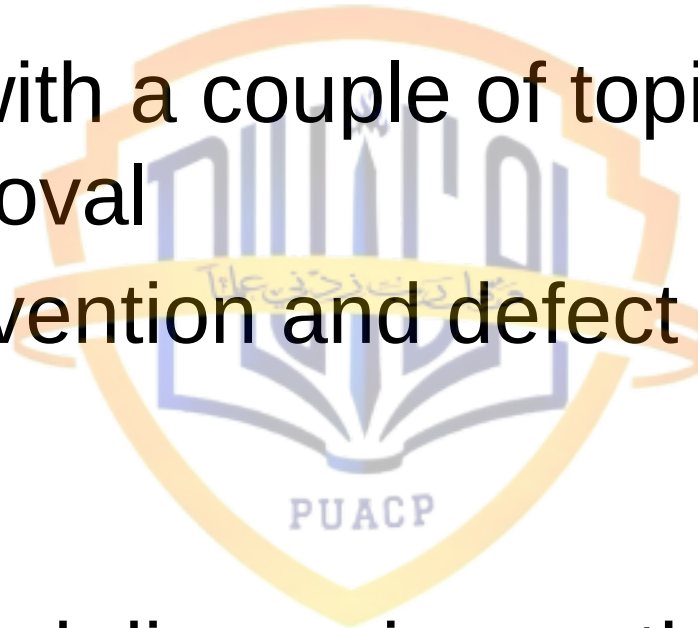


Software Defects - II

Lecture # 3

Today's Lecture

- Continue with a couple of topics related to defect removal
- Defect prevention and defect prevention techniques
- Have a good discussion on this topic before jumping directly on to the topic



Defect Repair Rates - 1

- Reported data on defect repair rates is not consistent
- Defect repair rates usually decline as cyclomatic and essential complexity increases
- Defect repair rates increase with experience in application, language, inspections, structured design and coding methods

Defect Repair Rates - 2

- Defect repair rates are higher for maintenance specialists than others during maintenance phase
- For coding errors, they correlate with comment density. IBM's study concluded that 18% comment density is ideal
- It also found, flow charts had no impact, but good error messages had great impact

Defect Seeding

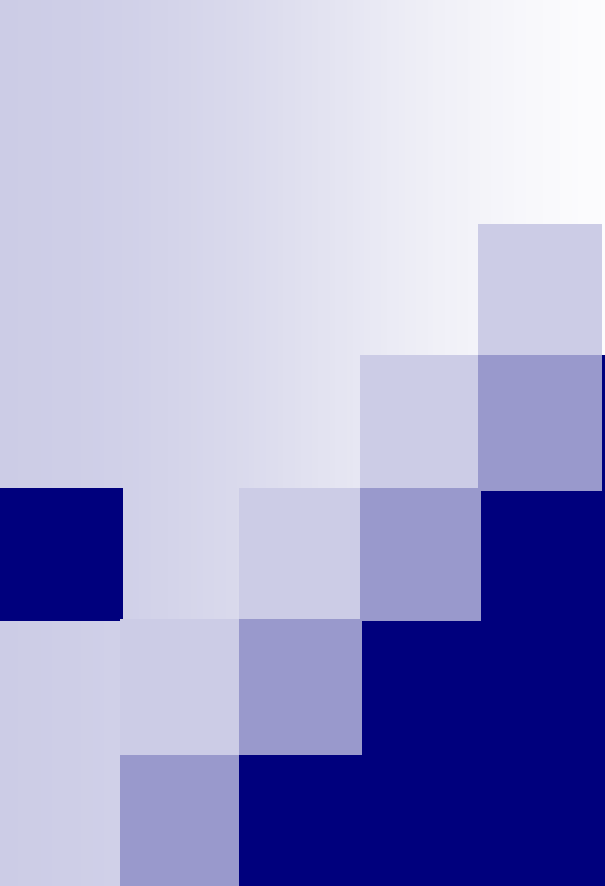
- Willful insertion of errors into a software deliverable prior to a review, inspection, or testing activity
- It is the quickest way of determining defect removal efficiency
- Considered unpleasant by many

Defect Severity Levels

- Most software defect tracking systems include a multi-tier “severity level” scale
- For example,
 - Severity level 1: total failure of application
 - Severity level 2: failure of major function(s)
 - Severity level 3: minor problem
 - Severity level 4: cosmetic problem

Defect Tracking

- It is important to use an accurate and automated defect tracking system
- Defect tracking tools
 - Tracking defects by severity level and by origin
 - Routing defects to appropriate repair facility
 - Keeping records of duplicate defects
 - Invalid defects
 - Repair information against defects



Defect Prevention



UQA

Defect Prevention

- We do not want defects or faults to enter our work products, requirements, design, code, or other documents
- We try to eliminate the error sources in defect prevention
- Defect prevention is very difficult to understand, study, and quantify

Philosophy of Defect Prevention - 1

- If human misconceptions are the error sources, education and training can help us remove these error sources
- If imprecise designs and implementations that deviate from product specifications or design intentions are the causes for faults, formal methods can help us prevent such deviations

Philosophy of Defect Prevention - 2

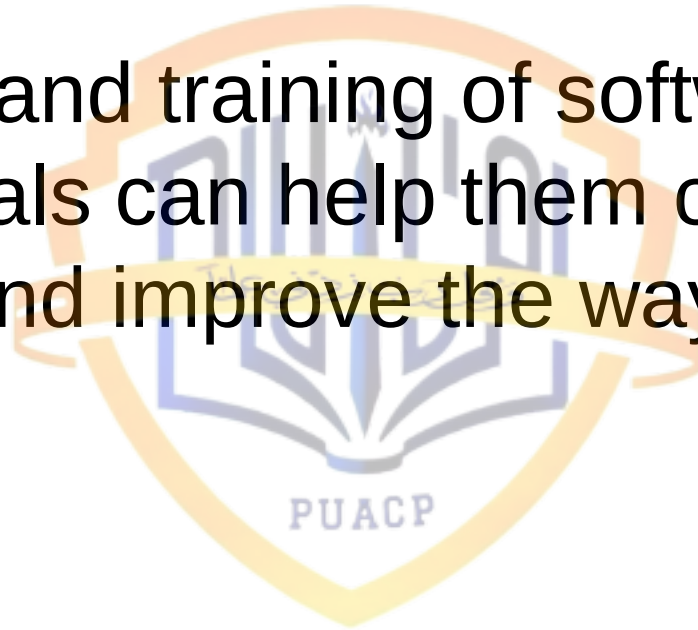
- If non-conformance to selected processes or standards is the problem that leads to fault injections, then process conformance or standard enforcement can help use prevent the injection of related faults
- If certain tools or technologies can reduce fault injections under similar environments, they should be adopted

Education and Training - 1

- Education and training provide people-based solutions for error source elimination
- The people factor is the most important factor that determines the quality and, ultimately, the success or failure of most software projects

Education and Training - 2

- Education and training of software professionals can help them control, manage, and improve the way they work



Focus of Education & Training

- Product and domain specific knowledge
- Software development knowledge and expertise
- Knowledge about Development methodology, technology, and tools
- Development process knowledge

Product and Domain Specific Knowledge

- If the people involved are not familiar with the product type or application domain, there is a good chance that wrong solutions will be implemented

Software Development Knowledge and Expertise

- Have good discussion on this



Knowledge about Development Methodology, Technology, and Tools

- Have good discussion on this



Development Process Knowledge

- Have good discussion on this



Formal Methods

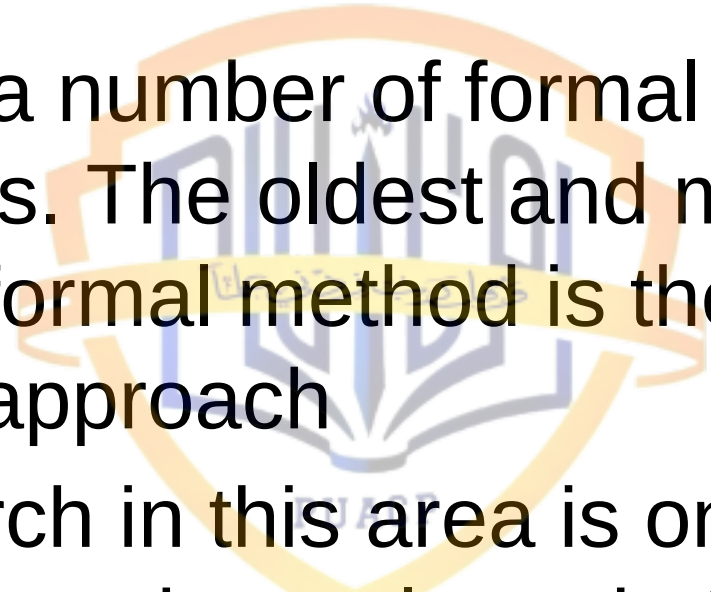
- Formal methods provide a way to eliminate certain error sources and to verify the absence of related faults
- Formal methods include
 - Formal specification
 - Formal verification

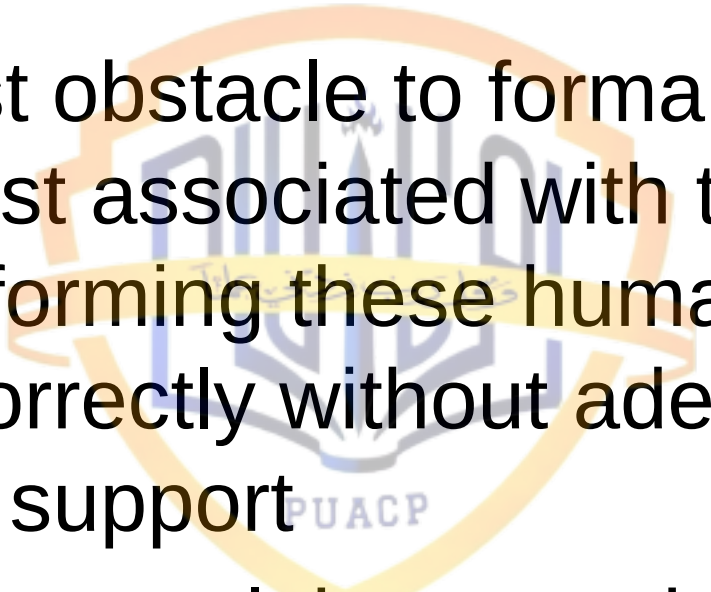
Formal Specification

- Formal specification is concerned with producing an unambiguous set of product specifications so that customer requirements, as well as environmental constraints and design intentions, are correctly reflected, thus reducing the chances of accidental fault injections

Formal Verifications

- Formal verification checks the conformance of software design or code against these formal specifications, thus ensuring that the software is fault free with respect to its formal specifications

- 
- 
- There are a number of formal method approaches. The oldest and most influential formal method is the so-called axiomatic approach
 - The research in this area is on-going and depending on the real need of the software applications, formal methods are being used

- 
- 
- The biggest obstacle to formal methods is the high cost associated with the difficult task of performing these human intensive activities correctly without adequate automated support
 - This fact also explains, to a degree, the increasing popularity of limited scope and semi-formal approaches

Other Defect Prevention Approaches

- Formal requirements analysis, i.e., JAD
- Formal risk-analysis early in the development
- Prototyping
- Structured programming methods
- Certified reusable design and code components

Software Defect Prevention



	Req. Defects	Design Defects	Code Defects	Document Defects	Perf. Defects
JAD	Excellent	Good	N/A	Fair	Poor
Prototypes	Excellent	Excellent	Fair	N/A	Excellent
Structured Methods	Fair	Good	Excellent	Fair	Fair
CASE Tools	Fair	Good	Fair	Fair	Fair
Blueprints / Reusable Code	Excellent	Excellent	Excellent	Excellent	Good
QFD	Good	Excellent	Fair	Poor	Good

Summary

- In today's lecture, we talked about defect prevention and defect prevention techniques
- We discussed the approaches to eliminating these defects

References

- Software Quality: Analysis and Guidelines for Success by Capers Jones
- Software Quality Engineering: Testing, Quality Assurance, and Quantifiable Improvement by Jeff Tian